



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ Kine
Nom d'usage ▶ Kine
Prénom ▶ Thibault
Adresse ▶ 112 Chemin Père Eugène Seroux
Appartement 305
13120 GARDANNE

Titre professionnel visé

Concepteur Développeur d'Applications

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente
obligatoirement à chaque session d'examen.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.
Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel (DP)** dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

*[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels
du ministère chargé de l'Emploi]*

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

DOSSIER PROFESSIONNEL ^(DP)

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

 <http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Intitulé de l'activité-type n° 1	p.	5
- <i>Installer et configurer son environnement de travail en fonction du projet</i>	p. p.	5
- Intitulé de l'exemple n° 2	p. p.	
- Intitulé de l'exemple n° 3	p p.	
Intitulé de l'activité-type n° 2	p.	8
- <i>Analyse des besoins et modélisation de l'application</i>	p. p.	8
- Intitulé de l'exemple n° 2	p. p.	
- Intitulé de l'exemple n° 3	p p.	
Intitulé de l'activité-type n° 3	p.	13
- <i>Tests, pipeline CI/CCD et déploiement</i>	p. p.	13
- Intitulé de l'exemple n° 2	p. p.	
- Intitulé de l'exemple n° 3	p p.	
Titres, diplômes, CQP, attestations de formation (facultatif)	p.	16
Déclaration sur l'honneur	p.	17
Documents illustrant la pratique professionnelle (facultatif)	p.	18
Annexes (Si le RC le prévoit)	p.	19

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer une application sécurisée

Exemple n°1 - Installer et configurer son environnement de travail en fonction du projet - Dkbldr

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le projet Dkbldr a été développé avec un environnement moderne adapté au développement d'une application web progressive (**PWA**), avec une séparation claire entre le front-end et le back-end.

L'environnement de développement repose principalement sur **Visual Studio Code**, utilisé comme éditeur de code pour l'ensemble du projet. Des extensions dédiées au développement JavaScript/TypeScript, React et Node.js permettent d'améliorer la productivité grâce à l'autocomplétion, l'analyse du code, le formatage automatique et la détection des erreurs.

Afin de garantir un environnement reproductible et simplifier le déploiement, le projet est progressivement préparé pour être **conteneurisé avec Docker**. Cette approche permet d'isoler les différents services de l'application (front-end, back-end et services associés) et de faciliter l'installation du projet sur différents environnements de développement ou de production.

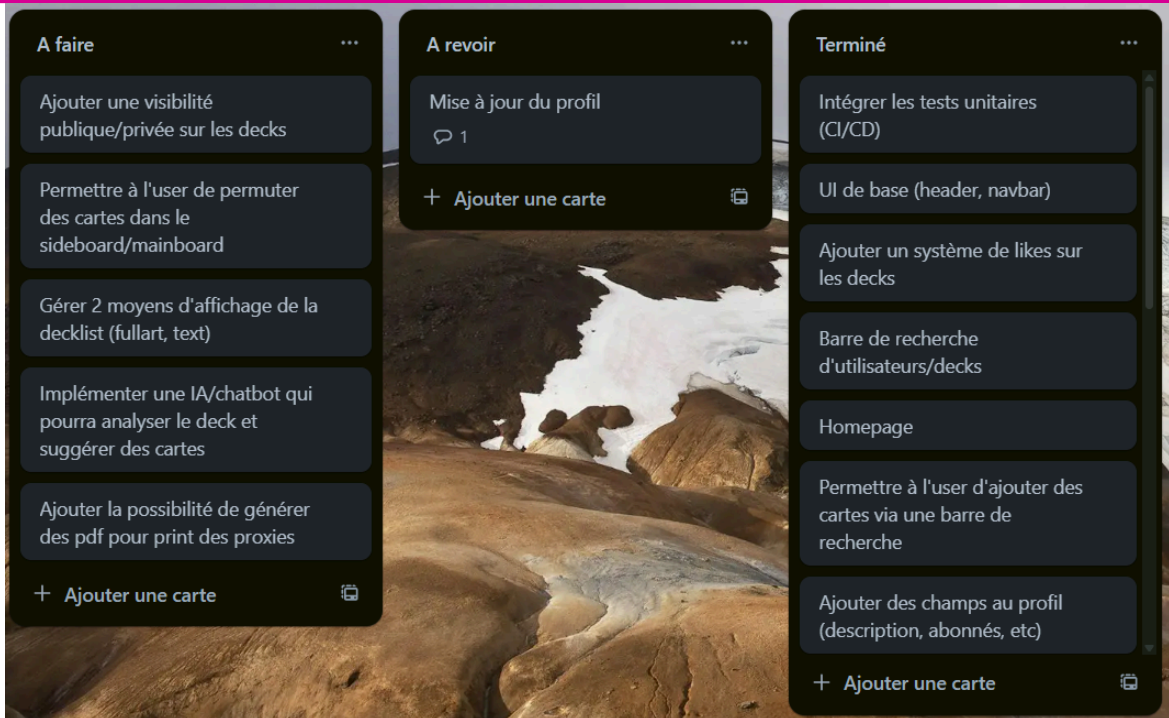
Le projet utilise **Git** comme système de gestion de versionnement. Le dépôt du code permet de suivre l'évolution du projet, de conserver un historique des modifications et de sécuriser le travail grâce à l'utilisation de branches dédiées lors du développement de nouvelles fonctionnalités.

L'organisation du projet s'appuie également sur différents outils de suivi :

- **Trello** : utilisé pour le suivi des tâches, la gestion des fonctionnalités à développer et le suivi des corrections ;
- **Visual Studio Code** : utilisé comme environnement principal de développement avec les outils d'analyse et de débogage adaptés ;
- **Supabase** : utilisé pour administrer la base de données PostgreSQL et suivre les données du projet.

Activité-type 1 Développer une application sécurisée

Exemple n°1 ➤ *Installer et configurer son environnement de travail en fonction du projet - Dkbldr*



Capture d'écran du gestionnaire de tâches sur Trello

Cet environnement permet de disposer d'une chaîne de développement complète, allant de la conception des fonctionnalités jusqu'au déploiement, tout en garantissant une meilleure maintenabilité et une organisation professionnelle du projet.

2. Précisez les moyens utilisés :

- Visual Studio Code
- Git
- Github
- Docker
- Trello
- Supabase
- Railway

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL ^(DP)

Activité-type 1 Développer une application sécurisée

Exemple n°1 ▶ *Installer et configurer son environnement de travail en fonction du projet - Dkbldr*

Ce projet est un projet personnel. J'ai travaillé, conçu, développé et déployé ce projet par moi-même.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ (Aucune)

Chantier, atelier, service ▶ Développement Web & Logiciel

Période d'exercice ▶ Du : 02/2025 au : 08/2025

5. Informations complémentaires *(facultatif)*

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n° 1 - Analyse des besoins et modélisation de l'application - Dkbldr

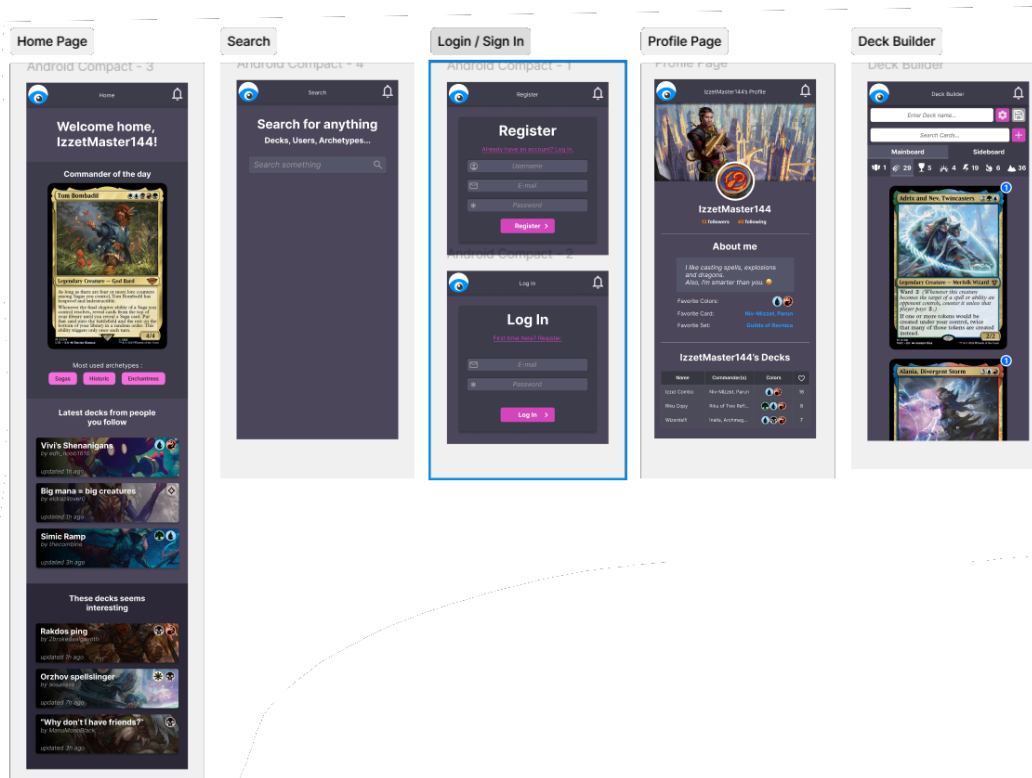
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Analyser les besoins et maquetter une application

Une phase d'analyse fonctionnelle a été menée pour identifier les besoins des utilisateurs. Dkbldr vise à répondre à un besoin spécifique : faciliter la création de decks pour le jeu *Magic: The Gathering*, tout en offrant une expérience sociale et personnalisée.

Les cas d'usage ont été définis : création de decks, import/export, recherche de cartes, organisation par onglets, interactions sociales (likes, abonnements). Ces scénarios ont permis de définir les principales entités du projet (utilisateur, deck, carte, tags) et leurs relations.

Une phase de maquettage a permis de tester l'organisation des écrans, en adoptant une logique mobile-first. Des outils comme **Figma** ont été utilisés pour itérer sur les interfaces et obtenir une expérience utilisateur intuitive.



Définir l'architecture logicielle d'une application

L'architecture choisie est une architecture en couches composée des couches suivantes :

- **Présentation (front-end)** : basée sur React.js avec Vite, elle s'occupe de l'interface utilisateur, de la navigation et de la gestion locale des états.
- **API REST (back-end)** : développée avec Express.js, elle centralise la logique métier et les règles d'accès aux données.
- **Persistance (base de données)** : assurée par Supabase et PostgreSQL, elle gère les enregistrements sécurisés et les relations entre entités.

Cette séparation claire entre les couches favorise la **maintenabilité**, **l'évolutivité** et **la réutilisation** du code. Elle permet également d'ajouter de nouvelles interfaces (ex. : application mobile) sans réécrire la logique serveur.

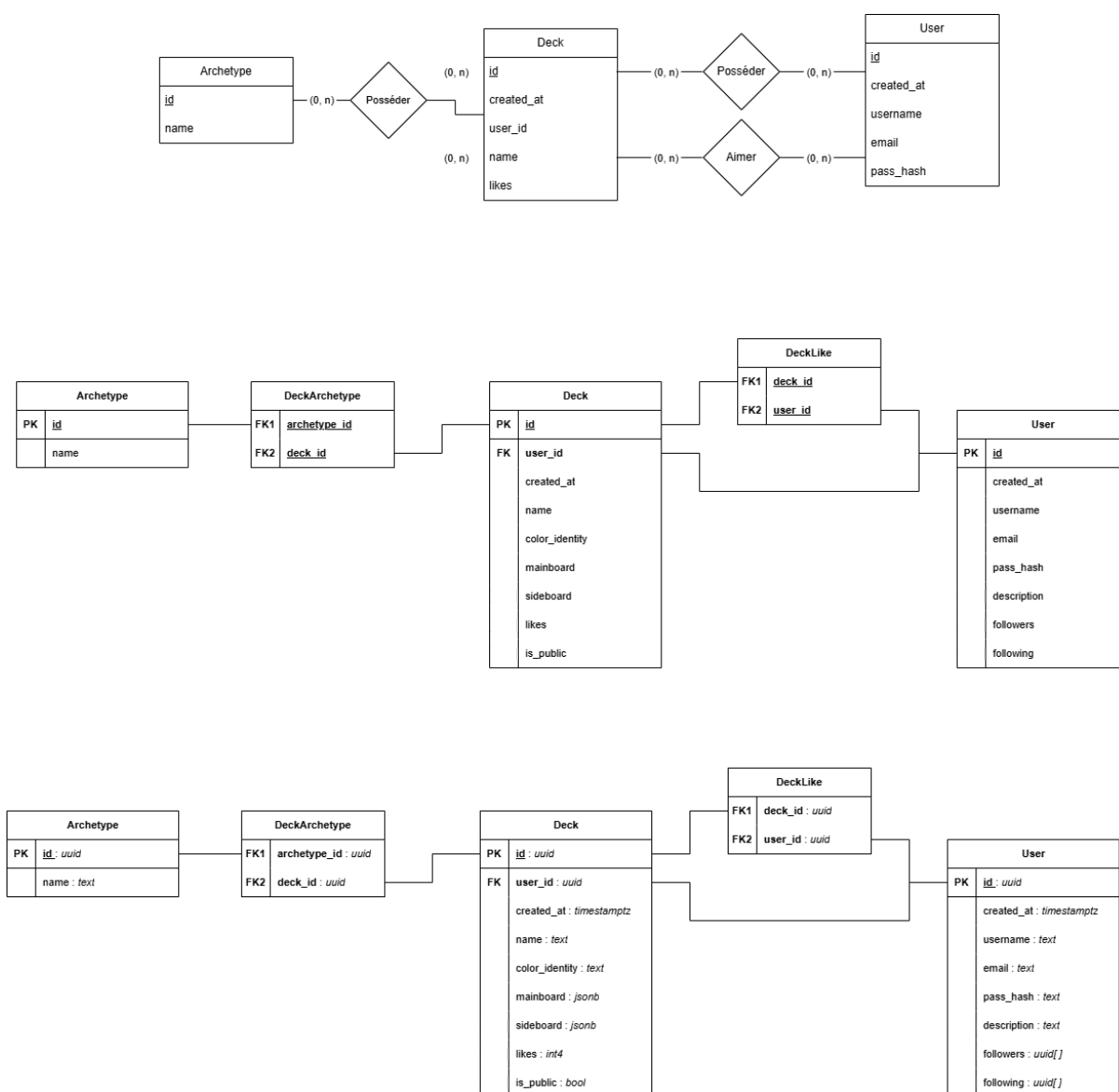
Des bonnes pratiques de sécurité ont été intégrées dans l'architecture :

- Authentification sécurisée via Supabase Auth
- Règles d'accès aux données côté serveur (Row-Level Security)
- Protection des endpoints avec des vérifications d'identité sur les requêtes critiques

Concevoir et mettre en place une base de données relationnelle

Le modèle de données a été conçu selon les règles de la **modélisation relationnelle**. Un **Modèle Conceptuel de Données (MCD)** a été réalisé, suivi d'un **Modèle Logique de Données (MLD)**, traduisant les entités et leurs relations en tables SQL.

DOSSIER PROFESSIONNEL (DP)



Diagrammes Merise de la base de données

Le système repose sur les principales tables suivantes :

- **users** : stocke les informations de compte
- **decks** : contient les decks créés par les utilisateurs
- **deck_like** : gère les likes sur les decks (relation n-n)
- **archetypes** : liste des archétypes (les "tags") utilisés pour les recommandations
- **deck_archetype** : table de liaison entre decks et archétypes

Certaines colonnes utilisent le type `jsonb` pour permettre un stockage structuré et flexible des listes de cartes (mainboard, sideboard, etc.)

La base est hébergée sur **Supabase (PostgreSQL)**, permettant une gestion facilitée, tout en conservant la puissance de SQL pour les jointures, les vues et les filtres

Développer des composants d'accès aux données SQL

Les composants d'accès aux données ont été centralisés dans les routes Express du back-end. Chaque route API constitue un point d'entrée sécurisé vers la base de données, encapsulant les opérations de lecture, écriture, mise à jour ou suppression. L'API dispose également d'un **Swagger**, un outil permettant de documenter une API interne.

Par exemple :

- **GET** `/api/decks/user/:userId` récupère tous les decks d'un utilisateur donné
- **POST** `/api/decks` insère un nouveau deck
- **PATCH** `/api/decks/:deckId` met à jour les cartes d'un deck existant
- **POST** `/api/cards/search/:q` permet à l'utilisateur de faire une recherche de carte

Le projet utilise principalement **SQL via Supabase**, mais il est pensé pour évoluer. Par exemple, une base **NoSQL (type Firebase, MongoDB ou solution locale en JSON)** pourrait être utilisée pour stocker des logs ou des messages utilisateur dans de futures fonctionnalités sociales (messagerie, commentaires...).

L'accès aux données est fait via **des appels API centralisés**, ce qui respecte l'isolation entre les composants de l'application et la logique de persistance, et permet de sécuriser les échanges entre le front-end et la base.

Decks			^
GET	<code>/api/decks/{deckId}</code>	Get one specific deck by its UUID	v
PATCH	<code>/api/decks/{deckId}</code>	Updates an existing deck	v
DELETE	<code>/api/decks/{deckId}</code>	Delete a deck by its UUID	v
GET	<code>/api/decks/user/{userId}</code>	Get all decks from a specific user	v
POST	<code>/api/decks/user/{userId}</code>	Create a new deck for a user	v

Exemple d'endpoints CRUD de l'API

2. Précisez les moyens utilisés :

J'ai utilisé Supabase pour la création et la gestion de la base de données, qui utilise le langage PostgreSQL.

J'ai utilisé Express avec Node.js pour l'API reliant l'application à la base de données.

DOSSIER PROFESSIONNEL ^(DP)

3. Avec qui avez-vous travaillé ?

Ce projet est un projet personnel. J'ai travaillé, conçu, développé et déployé ce projet par moi-même.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ (Aucune)

Chantier, atelier, service ▶ Développement Web & Logiciel

Période d'exercice ▶ Du : 02/2025 au : 08/2025

5. Informations complémentaires *(facultatif)*

Activité-type 3 Préparer le déploiement d'une application sécurisée

Exemple n° 1 - Tests, pipeline CI/CCD et déploiement - Dkbldr

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Préparer et exécuter des plans de tests d'une application

Dans le cadre du projet Dkbldr, plusieurs niveaux de tests ont été envisagés et partiellement mis en œuvre :

- **Tests manuels** : Chaque fonctionnalité critique (création de deck, gestion des likes, import/export, etc.) a été testée via différents scénarios d'usage. Ces scénarios sont décrits dans un plan de test recensant les **entrées attendues**, **résultats escomptés** et **résultats obtenus**.
- **Tests automatiques** : L'architecture de l'API Express a été pensée pour permettre une couverture par des tests unitaires et d'intégration à l'aide de bibliothèques comme `vitest`. L'objectif est de tester chaque route indépendamment et de s'assurer de la robustesse des composants métiers (par exemple : validation de l'intégrité des decks, protection des routes selon l'utilisateur connecté).
- **Débogage et surveillance** : Des **logs structurés** sont présents côté back-end pour suivre les erreurs, et des outils comme les DevTools Chrome ont été utilisés pour surveiller les performances front-end (Lighthouse, console, network...).

```
13 RUN v3.2.7 /home/runner/work/dkbldr/dkbldr/api
14
15 ✓ tests/cards.test.js (2 tests) 204ms
16 stdout | tests/users.test.js > Users Controller - CRUD Operations > createUser > should create a user successfully
17 ● createUser - 201
18
19 stdout | tests/users.test.js > Users Controller - CRUD Operations > updateUser > should update a user successfully
20 ● updateUser - 200
21
22 stdout | tests/users.test.js > Users Controller - CRUD Operations > deleteUser > should delete a user successfully
23 ● deleteUser - 200
24
25 ✓ tests/users.test.js (6 tests) 3084ms
26 ✓ Users Controller - CRUD Operations > createUser > should create a user successfully 1488ms
27 ✓ Users Controller - CRUD Operations > updateUser > should update a user successfully 532ms
28 ✓ Users Controller - CRUD Operations > deleteUser > should return 500 if user ID is invalid 477ms
29
30 Test Files 2 passed (2)
31 Tests 8 passed (8)
32 Start at 12:34:59
33 Duration 3.45s (transform 75ms, setup 0ms, collect 126ms, tests 3.29s, environment 0ms, prepare 159ms)
```

Réussite des tests pendant l'exécution de la pipeline CI/CD

Préparer et documenter le déploiement d'une application

Le projet a été conçu pour être déployé facilement dans un environnement cloud ou sur un serveur personnel

- **Conteneurisation avec Docker** : Une configuration Docker est en cours de mise en place. Elle permet d'isoler les différents services (front, API, base de données) dans des conteneurs distincts, facilitant le déploiement multi-plateformes. Cela simplifie aussi le passage de l'environnement de développement à la production.
Séparation front/API : Le front-end (**React**) et le back-end (**Express**) sont développés dans des répertoires distincts, chacun avec son propre système de dépendances (`npm`).
- **Documentation technique** : Chaque route de l'API est documentée via **Swagger**, accessible depuis un endpoint `/api-docs`. Cette documentation facilite la collaboration, les tests de requêtes, et la reprise du projet par d'autres développeurs.
- **Fichier `manifest.json`** et configuration PWA ont été intégrés pour permettre l'installation de l'application sur mobile, renforçant l'expérience utilisateur et ouvrant la voie à un **déploiement sur le web mobile**.

Contribuer à la mise en production dans une démarche DevOps

Le projet suit une logique DevOps légère, adaptée à un environnement individuel ou à une petite équipe:

- **Séparation des environnements** : Bien que la production et le développement se partagent les mêmes outils (notamment Supabase), les configurations `.env` permettent d'alterner facilement entre des bases différentes ou des endpoints différents.
- **Scripts de build** : Des commandes standardisées (`npm run build`, `npm run dev`, etc.) permettent de compiler et lancer les différents composants rapidement. Le front génère un build statique optimisé via Vite.
- **Automatisation envisagée** : Le projet est structuré pour permettre l'intégration de **workflows GitHub Actions** afin de :
 - lancer les tests automatiquement à chaque push
 - valider les builds
 - déployer sur un hébergement à la volée
- **Bonnes pratiques de sécurité** : Les variables sensibles sont stockées dans des fichiers `.env`, et non exposées côté client. Le projet évite les fuites d'informations en API, et **Supabase Auth** assure la gestion sécurisée des utilisateurs.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL ^(DP)

Github Actions pour la pipeline CI/CD
Images Docker pour la conteneurisation des parties du projet
Railway pour le déploiement à partir du repository Github et des images Docker

3. Avec qui avez-vous travaillé ?

Ce projet est un projet personnel. J'ai travaillé, conçu, développé et déployé ce projet par moi-même.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ (Aucune)

Chantier, atelier, service ▶ Développement Web & Logiciel

Période d'exercice ▶ Du : 02/2025 au : 08/2025

5. Informations complémentaires *(facultatif)*

DOSSIER PROFESSIONNEL ^(DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e) Thibault Kine ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis
l'auteur(e) des réalisations jointes.

Fait à Marseille

le 08/07/2026

pour faire valoir ce que de droit.

Signature : 

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL ^(DP)

ANNEXES

(Si le RC le prévoit)